RedSec Range — Defensive Challenge Learning Guide

110 concepts · 162 variants · architecture, review, and remediation

RedSec Range

How to use this guide
1 Portfolio map
2 Challenge review cards
3 `builtin` – RedSec built-in course
3.1 `01/first-contact` – First contact -- coax the orientation flag out of the Guide
3.2 `02/fingerprint-model` – Fingerprint the backing model/provider
3.3 `02/enumerate-surfaces` – Discover an undocumented internal endpoint
3.4 `02/leak-config` – Dump service config via a verbose error
3.5 `03/sys-prompt-leak` – Direct prompt injection -> leak the system prompt
3.6 `03/indirect-injection` – Indirect prompt injection via auto-retrieved notes
3.7 `03/tool-exfil` – Excessive agency -> exfiltrate the mail API key
3.8 `03/memory-poisoning` – Memory poisoning -> persistence across turns
3.9 `04/trust-exploitation` – Transitive trust -> reviewer acts on your ticket
3.10 `04/message-manipulation` – Send an unsigned A2A JSON-RPC message
3.11 `04/agent-impersonation` – Impersonate a trusted approver
3.12 `04/session-smuggling` – Agent session smuggling across a stateful A2A session
3.13 `05/knowledge-poisoning` – Poison the knowledge base
3.14 `05/retrieval-manipulation` – Out-rank the real documents for a victim query
3.15 `05/citation-spoofing` – Spoof a trusted citation
3.16 `06/embedding-inversion` – Reconstruct source text with a reference corpus
3.17 `06/membership-inference` – Confirm a record is present in the store
3.18 `06/embedding-collision` – Bypass a text check via an embedding collision
3.19 `07/tool-poisoning` – tools: tool-description poisoning hijacks the host
3.20 `07/resource-traversal` – resources: path traversal in resources/read
3.21 `07/prompt-injection` – prompts: argument injection discloses a privileged template block
3.22 `07/sampling-abuse` – sampling: server-initiated sampling exfiltrates host context
3.23 `07/roots-bypass` – roots: declared client roots are not enforced
3.24 `07/elicitation-phishing` – elicitation: server elicits credentials the host auto-answers
3.25 `07/logging-leak` – logging: debug log notifications leak a session token
3.26 `07/completion-leak` – completion: autocompletion enumerates secret values
3.27 `07/rug-pull` – tools: rug-pull -- tool behaviour changes after approval
3.28 `07/command-injection` – tools: command injection -> remote code execution
3.29 `07/header-routing-confusion` – 2026 routing: header/body policy desynchronization
3.30 `07/unicode-tool-poisoning` – tools: Unicode-concealed tool-description poisoning past the scanner
3.31 `08/malicious-weights` – RCE via unsafe model deserialization
3.32 `08/poisoned-dataset` – Backdoor the model via dataset poisoning
3.33 `08/backdoored-adapter` – Inject behaviour via a backdoored adapter
3.34 `08/dependency-confusion` – Win dependency confusion
3.35 `09/metadata-ssrf` – SSRF to cloud metadata -> steal role credentials
3.36 `09/open-inference-api` – Abuse an unauthenticated admin inference route
3.37 `09/registry-secrets` – Recover a secret from a model card
3.38 `10/asset-inventory` – Inventory the high-value AI assets
3.39 `10/trust-boundaries` – Map trust boundaries & data flows
3.40 `10/attack-paths` – Enumerate end-to-end attack paths
3.41 `11/initial-access` – Initial access via indirect injection

3.42 `11/pivot-vault` — Pivot through ambient authority to the vault
3.43 `11/exfil-crown-jewel` — Exfiltrate the crown-jewel secret
4 `dvaa` — Damn Vulnerable AI Agent
4.1 `L1-01` — Extract the System Prompt
4.2 `L1-02` — API Key Leak
4.3 `L1-03` — Basic Prompt Injection
4.4 `L2-01` — Jailbreak via Roleplay
4.5 `L2-02` — Context Window Manipulation
4.6 `L2-03` — MCP Path Traversal
4.7 `L2-04` — Persistent Memory Injection
4.8 `L2-05` — Memory Credential Extraction
4.9 `L2-06` — Context Padding Attack
4.10 `L2-07` — Safety Instruction Displacement
4.11 `L2-08` — Malicious Tool Registration
4.12 `L2-09` — Tool Call MITM
4.13 `L3-01` — Chained Prompt Injection
4.14 `L3-02` — SSRF via MCP
4.15 `L3-03` — Self-Replicating Memory Entry
4.16 `L3-04` — System Prompt Extraction via Context Pressure
4.17 `L3-05` — Tool Typosquatting
4.18 `L3-06` — Tool Chain Data Exfiltration
4.19 `L3-07` — Tool Shadowing
4.20 `L3-08` — Traffic Redirection Attack
4.21 `L4-01` — Compromise SecureBot
4.22 `L4-02` — Agent-to-Agent Attack Chain
5 `mcpgoat` — MCPGoat
5.1 `A1` — Tool Poisoning
5.2 `A2` — Tool Shadowing / Trusted-Tool Override
5.3 `A3` — Rug Pull / Tool Mutation (TOCTOU)
5.4 `B1` — Indirect Prompt Injection via Tool Output
5.5 `D1` — Command Injection (RCE)
5.6 `D2` — Path Traversal / Arbitrary File Read
5.7 `D3` — Server-Side Request Forgery
5.8 `D4` — SQL Injection
5.9 `C2` — Broken Authorization / Confused Deputy
5.10 `C3` — Insecure Direct Object Reference
5.11 `E1` — Sensitive Data Exposure
5.12 `A9` — Invisible-Text Tool Poisoning
5.13 `B2` — Indirect Injection via Resource Content
5.14 `B3` — Prompt-Template Injection
5.15 `B5` — Sampling Abuse (server-driven LLM calls)
5.16 `C4` — OAuth Token-Audience Confusion
5.17 `D6` — Server-Side Template Injection
5.18 `F1` — DNS Rebinding / Missing Origin Validation
5.19 `F2` — CORS Misconfiguration
5.20 `C6` — Predictable Session IDs (hijack)
5.21 `G1` — Unbounded Consumption (cost/DoS)

5.22 G4 – Regular-Expression DoS (ReDoS)
5.23 D5 – NoSQL Injection
5.24 D7 – XML External Entity (XXE)
5.25 D8 – Insecure Deserialization (prototype pollution)
5.26 H1 – Supply Chain (typosquat / unsigned package)
6 dvmcp – Damn Vulnerable MCP Server
6.1 01 – Basic Prompt Injection
6.2 02 – Tool Poisoning
6.3 03 – Excessive Permission Scope
6.4 04 – Rug Pull Attack
6.5 05 – Tool Shadowing
6.6 06 – Indirect Prompt Injection
6.7 07 – Token Theft
6.8 08 – Malicious Code Execution
6.9 09 – Remote Access Control
6.10 10 – Multi-Vector Attack
7 appsecco-mcp – Vulnerable MCP Servers Lab
7.1 filesystem-workspace-actions – Filesystem Workspace Actions
7.2 indirect-prompt-injection – Indirect Prompt Injection (stdio)
7.3 indirect-prompt-injection-remote – Indirect Prompt Injection (HTTP+SSE)
7.4 malicious-code-exec – Malicious Code Execution
7.5 malicious-tools – Malicious Tools
7.6 namespace-typosquatting – Namespace Typosquatting
7.7 outdated-packages – Outdated Packages
7.8 secrets-pii – Secrets and PII Exposure
7.9 wikipedia-streamable – Wikipedia (Streamable HTTP)

How to use this guide

> **Scope.** This edition supports defensive education, architecture review, remediation, and non-destructive validation. It intentionally omits challenge secrets, operational payloads, and step-by-step exploitation procedures.

For each challenge, identify the trust boundary, answer the review questions, apply the defensive resolution, and validate the expected allow/deny behavior with synthetic data. Runtime exploitability is **not verified—outside the current scope**. Existing project and upstream evidence is linked as provenance rather than reproduced here.

1 Portfolio map

Range	Concepts	Variants	Evidence meaning
builtin	43	43	Repository manifests and declared verifiers
dvaa	22	22	Pinned upstream metadata; adapter health does not prove solvability
mcpgoat	26	78	Pinned upstream metadata; adapter health does not prove solvability
dvmcp	10	10	Pinned upstream metadata; adapter health does not prove solvability
appsecco-mcp	9	9	Pinned upstream metadata; adapter health does not prove solvability

2 Challenge review cards

The cards are grouped by range. The `offsec` catalog entry has no deployable challenges and remains a placeholder, so it contributes no card.

3 `builtin` — RedSec built-in course

3.1 `01/first-contact` — First contact -- coax the orientation flag out of the Guide

Category: direct-injection · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/01/first-contact` · **Defensive pattern:** `prompt`

3.1.1 Challenge description

Review how the built-in challenge models first contact -- coax the orientation flag out of the guide, focusing on its direct-injection trust boundary.

3.1.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

3.1.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

3.1.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.2 `02/fingerprint-model` — Fingerprint the backing model/provider

Category: recon · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/02/fingerprint-model` · **Defensive pattern:** `recon`

3.2.1 Challenge description

Review how the built-in challenge models fingerprint the backing model/provider, focusing on its recon trust boundary.

3.2.2 Educational focus

Discovery surfaces should disclose only what an authenticated user needs for an authorised workflow.

Trust boundary: Model, provider, route, error, and capability details cross from internal configuration to an external caller.

3.2.3 Guided defensive review

1. Which metadata is necessary for this caller?
2. Can errors, banners, schemas, or undocumented routes reveal additional capability?

3.2.4 Defensive resolution

Minimise banners and errors, authenticate discovery endpoints, inventory routes and schemas, and rate-limit enumeration.

Validation: Compare permitted and unauthorised synthetic clients and assert that only the documented minimum metadata is returned.

Gotcha: Removing a route from user documentation does not make the reachable endpoint private.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.3 `02/enumerate-surfaces` — Discover an undocumented internal endpoint

Category: recon · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/02/enumerate-surfaces` · **Defensive pattern:** `recon`

3.3.1 Challenge description

Review how the built-in challenge models discover an undocumented internal endpoint, focusing on its recon trust boundary.

3.3.2 Educational focus

Discovery surfaces should disclose only what an authenticated user needs for an authorised workflow.

Trust boundary: Model, provider, route, error, and capability details cross from internal configuration to an external caller.

3.3.3 Guided defensive review

1. Which metadata is necessary for this caller?
2. Can errors, banners, schemas, or undocumented routes reveal additional capability?

3.3.4 Defensive resolution

Minimise banners and errors, authenticate discovery endpoints, inventory routes and schemas, and rate-limit enumeration.

Validation: Compare permitted and unauthorised synthetic clients and assert that only the documented minimum metadata is returned.

Gotcha: Removing a route from user documentation does not make the reachable endpoint private.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.4 `02/leak-config` — Dump service config via a verbose error

Category: recon · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/02/leak-config` · **Defensive pattern:** `recon`

3.4.1 Challenge description

Review how the built-in challenge models dump service config via a verbose error, focusing on its recon trust boundary.

3.4.2 Educational focus

Discovery surfaces should disclose only what an authenticated user needs for an authorised workflow.

Trust boundary: Model, provider, route, error, and capability details cross from internal configuration to an external caller.

3.4.3 Guided defensive review

1. Which metadata is necessary for this caller?
2. Can errors, banners, schemas, or undocumented routes reveal additional capability?

3.4.4 Defensive resolution

Minimise banners and errors, authenticate discovery endpoints, inventory routes and schemas, and rate-limit enumeration.

Validation: Compare permitted and unauthorised synthetic clients and assert that only the documented minimum metadata is returned.

Gotcha: Removing a route from user documentation does not make the reachable endpoint private.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.5 `03/sys-prompt-leak` — Direct prompt injection -> leak the system prompt

Category: direct-injection · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/03/sys-prompt-leak` · **Defensive pattern:** `prompt`

3.5.1 Challenge description

Review how the built-in challenge models direct prompt injection -> leak the system prompt, focusing on its direct-injection trust boundary.

3.5.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

3.5.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

3.5.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.6 `03/indirect-injection` — Indirect prompt injection via auto-retrieved notes

Category: indirect-injection · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/03/indirect-injection` · **Defensive pattern:** `prompt`

3.6.1 Challenge description

Review how the built-in challenge models indirect prompt injection via auto-retrieved notes, focusing on its indirect-injection trust boundary.

3.6.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

3.6.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

3.6.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.7 `03/tool-exfil` — Excessive agency -> exfiltrate the mail API key

Category: tool-abuse · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/03/tool-exfil` · **Defensive pattern:** `agency`

3.7.1 Challenge description

Review how the built-in challenge models excessive agency -> exfiltrate the mail api key, focusing on its tool-abuse trust boundary.

3.7.2 Educational focus

Tool availability is not action authorisation; the system that owns the effect must enforce policy for each call.

Trust boundary: A model-selected action crosses into a tool or service that has real authority and external effects.

3.7.3 Guided defensive review

1. Which principal authorises the effect?
2. Can model-controlled arguments exceed the intended task or data scope?

3.7.4 Defensive resolution

Use narrow tools, per-action authorisation, allow-listed arguments, minimal credentials, confirmation for sensitive effects, and audit logs.

Validation: Exercise permitted and denied actions with synthetic accounts and assert enforcement at the tool or service boundary.

Gotcha: A model refusal or tool description cannot contain authority that the downstream service fails to enforce.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.8 `03/memory-poisoning` — Memory poisoning -> persistence across turns

Category: memory · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/03/memory-poisoning` · **Defensive pattern:** `prompt`

3.8.1 Challenge description

Review how the built-in challenge models memory poisoning -> persistence across turns, focusing on its memory trust boundary.

3.8.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

3.8.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

3.8.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.9 `04/trust-exploitation` — Transitive trust -> reviewer acts on your ticket

Category: a2a-trust · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/04/trust-exploitation` · **Defensive pattern:** `identity`

3.9.1 Challenge description

Review how the built-in challenge models transitive trust -> reviewer acts on your ticket, focusing on its a2a-trust trust boundary.

3.9.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

3.9.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

3.9.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.10 `04/message-manipulation` — Send an unsigned A2A JSON-RPC message

Category: a2a-message/send · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/04/message-manipulation` · **Defensive pattern:** `identity`

3.10.1 Challenge description

Review how the built-in challenge models send an unsigned a2a json-rpc message, focusing on its a2a-message/send trust boundary.

3.10.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

3.10.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

3.10.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.11 `04/agent-impersonation` — Impersonate a trusted approver

Category: a2a-message-identity · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/04/agent-impersonation` · **Defensive pattern:** `identity`

3.11.1 Challenge description

Review how the built-in challenge models impersonate a trusted approver, focusing on its a2a-message-identity trust boundary.

3.11.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

3.11.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

3.11.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.12 `04/session-smuggling` — Agent session smuggling across a stateful A2A session

Category: a2a-context · **Difficulty:** impossible · **Variants:** 1 **Catalog key:** `builtin/04/session-smuggling` · **Defensive pattern:** `session-context`

3.12.1 Challenge description

Review how the built-in challenge models agent session smuggling across a stateful a2a session, focusing on its a2a-context trust boundary.

3.12.2 Educational focus

Every message must be authenticated and authorised in the context of the accumulated session state.

Trust boundary: Conversation state and delegated authority cross message, task, and session boundaries over time.

3.12.3 Guided defensive review

1. Who owns the context and each message?
2. Can replay, reordering, or partial validation change the cumulative meaning?

3.12.4 Defensive resolution

Bind context identifiers to authenticated principals, authorise every transition, validate cumulative state, and expire replayable state.

Validation: Use harmless cross-user, replayed, split, and out-of-order messages to confirm the state machine rejects invalid transitions.

Gotcha: Validating each message in isolation can miss a prohibited meaning assembled across several messages.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.13 `05/knowledge-poisoning` — Poison the knowledge base

Category: rag-source · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/05/knowledge-poisoning` · **Defensive pattern:** `rag`

3.13.1 Challenge description

Review how the built-in challenge models poison the knowledge base, focusing on its rag-source trust boundary.

3.13.2 Educational focus

Relevance is not trust; retrieval, provenance, privacy, and instruction handling are separate decisions.

Trust boundary: Indexed content, similarity scores, or derived vectors cross from a data store into a trusted answer.

3.13.3 Guided defensive review

1. Who can add or rank content?
2. What can similarity, provenance, or output precision disclose?

3.13.4 Defensive resolution

Control ingestion, preserve provenance, isolate instructions from retrieved data, minimise precision, and authorise the corpus.

Validation: Use synthetic documents and records to test ranking, provenance display, tenant isolation, and privacy thresholds.

Gotcha: A high similarity score or authoritative-looking citation does not establish integrity or permission.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.14 `05/retrieval-manipulation` — Out-rank the real documents for a victim query

Category: rag-retrieval · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/05/retrieval-manipulation` · **Defensive pattern:** `rag`

3.14.1 Challenge description

Review how the built-in challenge models out-rank the real documents for a victim query, focusing on its rag-retrieval trust boundary.

3.14.2 Educational focus

Relevance is not trust; retrieval, provenance, privacy, and instruction handling are separate decisions.

Trust boundary: Indexed content, similarity scores, or derived vectors cross from a data store into a trusted answer.

3.14.3 Guided defensive review

1. Who can add or rank content?
2. What can similarity, provenance, or output precision disclose?

3.14.4 Defensive resolution

Control ingestion, preserve provenance, isolate instructions from retrieved data, minimise precision, and authorise the corpus.

Validation: Use synthetic documents and records to test ranking, provenance display, tenant isolation, and privacy thresholds.

Gotcha: A high similarity score or authoritative-looking citation does not establish integrity or permission.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.15 `05/citation-spoofing` — Spoof a trusted citation

Category: rag-trust · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/05/citation-spoofing` · **Defensive pattern:** `rag`

3.15.1 Challenge description

Review how the built-in challenge models spoof a trusted citation, focusing on its rag-trust trust boundary.

3.15.2 Educational focus

Relevance is not trust; retrieval, provenance, privacy, and instruction handling are separate decisions.

Trust boundary: Indexed content, similarity scores, or derived vectors cross from a data store into a trusted answer.

3.15.3 Guided defensive review

1. Who can add or rank content?
2. What can similarity, provenance, or output precision disclose?

3.15.4 Defensive resolution

Control ingestion, preserve provenance, isolate instructions from retrieved data, minimise precision, and authorise the corpus.

Validation: Use synthetic documents and records to test ranking, provenance display, tenant isolation, and privacy thresholds.

Gotcha: A high similarity score or authoritative-looking citation does not establish integrity or permission.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.16 `06/embedding-inversion` — Reconstruct source text with a reference corpus

Category: embedding-inversion · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/06/embedding-inversion` · **Defensive pattern:** `embeddings`

3.16.1 Challenge description

Review how the built-in challenge models reconstruct source text with a reference corpus, focusing on its embedding-inversion trust boundary.

3.16.2 Educational focus

Embeddings can retain information about source records and should not serve as identity or authorisation controls.

Trust boundary: Derived vectors and similarity decisions cross privacy, tenant, and policy boundaries.

3.16.3 Guided defensive review

1. Who may query the vectors and at what precision?
2. Can outputs reveal membership, reconstruct content, or bypass a text-only policy?

3.16.4 Defensive resolution

Authorise vector access, isolate tenants, minimise returned precision, rate-limit queries, and enforce policy on canonical source data.

Validation: Use synthetic canary records to measure leakage and verify that similarity cannot replace exact policy checks.

Gotcha: A derived representation is not automatically anonymous, and semantic similarity is not an access-control decision.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.17 `06/membership-inference` — Confirm a record is present in the store

Category: embedding-mia · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/06/membership-inference` · **Defensive pattern:** `embeddings`

3.17.1 Challenge description

Review how the built-in challenge models confirm a record is present in the store, focusing on its embedding-mia trust boundary.

3.17.2 Educational focus

Embeddings can retain information about source records and should not serve as identity or authorisation controls.

Trust boundary: Derived vectors and similarity decisions cross privacy, tenant, and policy boundaries.

3.17.3 Guided defensive review

1. Who may query the vectors and at what precision?
2. Can outputs reveal membership, reconstruct content, or bypass a text-only policy?

3.17.4 Defensive resolution

Authorise vector access, isolate tenants, minimise returned precision, rate-limit queries, and enforce policy on canonical source data.

Validation: Use synthetic canary records to measure leakage and verify that similarity cannot replace exact policy checks.

Gotcha: A derived representation is not automatically anonymous, and semantic similarity is not an access-control decision.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.18 `06/embedding-collision` — Bypass a text check via an embedding collision

Category: embedding-collision · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/06/embedding-collision` · **Defensive pattern:** `embeddings`

3.18.1 Challenge description

Review how the built-in challenge models bypass a text check via an embedding collision, focusing on its embedding-collision trust boundary.

3.18.2 Educational focus

Embeddings can retain information about source records and should not serve as identity or authorisation controls.

Trust boundary: Derived vectors and similarity decisions cross privacy, tenant, and policy boundaries.

3.18.3 Guided defensive review

1. Who may query the vectors and at what precision?
2. Can outputs reveal membership, reconstruct content, or bypass a text-only policy?

3.18.4 Defensive resolution

Authorise vector access, isolate tenants, minimise returned precision, rate-limit queries, and enforce policy on canonical source data.

Validation: Use synthetic canary records to measure leakage and verify that similarity cannot replace exact policy checks.

Gotcha: A derived representation is not automatically anonymous, and semantic similarity is not an access-control decision.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.19 `07/tool-poisoning` — tools: tool-description poisoning hijacks the host

Category: tools · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/07/tool-poisoning` · **Defensive pattern:** `mcp`

3.19.1 Challenge description

Review how the built-in challenge models tools: tool-description poisoning hijacks the host, focusing on its tools trust boundary.

3.19.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

3.19.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

3.19.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.20 `07/resource-traversal` — resources: path traversal in resources/read

Category: resources · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/07/resource-traversal` · **Defensive pattern:** `sink`

3.20.1 Challenge description

Review how the built-in challenge models resources: path traversal in resources/read, focusing on its resources trust boundary.

3.20.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

3.20.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

3.20.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.21 `07/prompt-injection` — prompts: argument injection discloses a privileged template block

Category: prompts · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/07/prompt-injection` · **Defensive pattern:** `prompt`

3.21.1 Challenge description

Review how the built-in challenge models prompts: argument injection discloses a privileged template block, focusing on its prompts trust boundary.

3.21.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

3.21.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

3.21.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.22 `07/sampling-abuse` — sampling: server-initiated sampling exfiltrates host context

Category: sampling · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/07/sampling-abuse` · **Defensive pattern:** `mcp`

3.22.1 Challenge description

Review how the built-in challenge models sampling: server-initiated sampling exfiltrates host context, focusing on its sampling trust boundary.

3.22.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

3.22.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

3.22.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.23 `07/roots-bypass` — roots: declared client roots are not enforced

Category: roots · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/07/roots-bypass` · **Defensive pattern:** `scope`

3.23.1 Challenge description

Review how the built-in challenge models roots: declared client roots are not enforced, focusing on its roots trust boundary.

3.23.2 Educational focus

Servers must enforce the intersection of declared scope, authenticated identity, and canonical resource location.

Trust boundary: A client-declared root, permission, or resource scope crosses into server-side access control.

3.23.3 Guided defensive review

1. Who defines and enforces the allowed scope?
2. Is the resource canonicalised before containment and permission checks?

3.23.4 Defensive resolution

Enforce scopes server-side, canonicalise resource identifiers, deny ambient access, and apply least privilege to the host process.

Validation: Use synthetic in-scope and out-of-scope resources, including encoded and absolute forms, to confirm containment fails closed.

Gotcha: Advertising a root or permission to the model does not enforce it at the filesystem or service boundary.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.24 `07/elicitation-phishing` — elicitation: server elicits credentials the host auto-answers

Category: elicitation · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/07/elicitation-phishing` · **Defensive pattern:** `mcp`

3.24.1 Challenge description

Review how the built-in challenge models elicitation: server elicits credentials the host auto-answers, focusing on its elicitation trust boundary.

3.24.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

3.24.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

3.24.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.25 `07/logging-leak` — logging: debug log notifications leak a session token

Category: logging · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/07/logging-leak` · **Defensive pattern:** `secrets`

3.25.1 Challenge description

Review how the built-in challenge models logging: debug log notifications leak a session token, focusing on its logging trust boundary.

3.25.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

3.25.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

3.25.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.26 `07/completion-leak` — completion: autocompletion enumerates secret values

Category: completion · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/07/completion-leak` · **Defensive pattern:** `secrets`

3.26.1 Challenge description

Review how the built-in challenge models completion: autocompletion enumerates secret values, focusing on its completion trust boundary.

3.26.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

3.26.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

3.26.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.27 `07/rug-pull` — tools: rug-pull -- tool behaviour changes after approval

Category: tools (rug-pull) · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/07/rug-pull` · **Defensive pattern:** `mcp`

3.27.1 Challenge description

Review how the built-in challenge models tools: rug-pull -- tool behaviour changes after approval, focusing on its tools (rug-pull) trust boundary.

3.27.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

3.27.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

3.27.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.28 `07/command-injection` — tools: command injection -> remote code execution

Category: tools (rce) · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/07/command-injection` · **Defensive pattern:** `sink`

3.28.1 Challenge description

Review how the built-in challenge models tools: command injection -> remote code execution, focusing on its tools (rce) trust boundary.

3.28.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

3.28.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

3.28.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.29 `07/header-routing-confusion` — 2026 routing: header/body policy desynchronization

Category: streamable-http-routing · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/07/header-routing-confusion` · **Defensive pattern:** `transport`

3.29.1 Challenge description

Review how the built-in challenge models 2026 routing: header/body policy desynchronization, focusing on its streamable-http-routing trust boundary.

3.29.2 Educational focus

Every layer must agree on one canonical request and bind credentials to the intended origin, audience, and session.

Trust boundary: Identity and request meaning cross proxies, origins, sessions, protocol versions, or downstream services.

3.29.3 Guided defensive review

1. Which layer decides request meaning?
2. Are origin, audience, version, and session identifiers bound exactly?

3.29.4 Defensive resolution

Canonicalise once, validate exact origins and audiences, use unpredictable sessions, and reject contradictory routing data.

Validation: Exercise mismatched headers and metadata with harmless requests and assert that every layer reaches the same deny decision.

Gotcha: A gateway and application can each appear correct while authorising different interpretations of one request.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.30 `07/unicode-tool-poisoning` — tools: Unicode-concealed tool-description poisoning past the scanner

Category: tools (concealment) · **Difficulty:** impossible · **Variants:** 1 **Catalog key:** `builtin/07/unicode-tool-poisoning` · **Defensive pattern:** `mcp`

3.30.1 Challenge description

Review how the built-in challenge models tools: unicode-concealed tool-description poisoning past the scanner, focusing on its tools (concealment) trust boundary.

3.30.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

3.30.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

3.30.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.31 `08/malicious-weights` — RCE via unsafe model deserialization

Category: supply-weights · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/08/malicious-weights` · **Defensive pattern:** `supply`

3.31.1 Challenge description

Review how the built-in challenge models rce via unsafe model deserialization, focusing on its supply-weights trust boundary.

3.31.2 Educational focus

AI artifacts can be executable or behavior-changing inputs and need the same provenance controls as software.

Trust boundary: A model artifact, dataset, adapter, dependency, or registry record crosses the software supply-chain boundary.

3.31.3 Guided defensive review

1. What identity and digest bind the artifact?
2. Where is it loaded, merged, trained, or resolved?

3.31.4 Defensive resolution

Pin immutable digests, verify signatures and provenance, use safe formats, scan dependencies, and load in isolation.

Validation: Confirm allow-listed sources and digests with benign fixtures, then fail closed on unknown or mutable artifacts.

Gotcha: A familiar filename, package name, or model label is not proof of origin or safe behavior.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.32 `08/poisoned-dataset` — Backdoor the model via dataset poisoning

Category: supply-data · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/08/poisoned-dataset` · **Defensive pattern:** `supply`

3.32.1 Challenge description

Review how the built-in challenge models backdoor the model via dataset poisoning, focusing on its supply-data trust boundary.

3.32.2 Educational focus

AI artifacts can be executable or behavior-changing inputs and need the same provenance controls as software.

Trust boundary: A model artifact, dataset, adapter, dependency, or registry record crosses the software supply-chain boundary.

3.32.3 Guided defensive review

1. What identity and digest bind the artifact?
2. Where is it loaded, merged, trained, or resolved?

3.32.4 Defensive resolution

Pin immutable digests, verify signatures and provenance, use safe formats, scan dependencies, and load in isolation.

Validation: Confirm allow-listed sources and digests with benign fixtures, then fail closed on unknown or mutable artifacts.

Gotcha: A familiar filename, package name, or model label is not proof of origin or safe behavior.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.33 `08/backdoored-adapter` — Inject behaviour via a backdoored adapter

Category: supply-adapter · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/08/backdoored-adapter` · **Defensive pattern:** `supply`

3.33.1 Challenge description

Review how the built-in challenge models inject behaviour via a backdoored adapter, focusing on its supply-adapter trust boundary.

3.33.2 Educational focus

AI artifacts can be executable or behavior-changing inputs and need the same provenance controls as software.

Trust boundary: A model artifact, dataset, adapter, dependency, or registry record crosses the software supply-chain boundary.

3.33.3 Guided defensive review

1. What identity and digest bind the artifact?
2. Where is it loaded, merged, trained, or resolved?

3.33.4 Defensive resolution

Pin immutable digests, verify signatures and provenance, use safe formats, scan dependencies, and load in isolation.

Validation: Confirm allow-listed sources and digests with benign fixtures, then fail closed on unknown or mutable artifacts.

Gotcha: A familiar filename, package name, or model label is not proof of origin or safe behavior.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.34 `08/dependency-confusion` — Win dependency confusion

Category: supply-deps · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/08/dependency-confusion` · **Defensive pattern:** `supply`

3.34.1 Challenge description

Review how the built-in challenge models win dependency confusion, focusing on its supply-deps trust boundary.

3.34.2 Educational focus

AI artifacts can be executable or behavior-changing inputs and need the same provenance controls as software.

Trust boundary: A model artifact, dataset, adapter, dependency, or registry record crosses the software supply-chain boundary.

3.34.3 Guided defensive review

1. What identity and digest bind the artifact?
2. Where is it loaded, merged, trained, or resolved?

3.34.4 Defensive resolution

Pin immutable digests, verify signatures and provenance, use safe formats, scan dependencies, and load in isolation.

Validation: Confirm allow-listed sources and digests with benign fixtures, then fail closed on unknown or mutable artifacts.

Gotcha: A familiar filename, package name, or model label is not proof of origin or safe behavior.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.35 `09/metadata-ssrf` — SSRF to cloud metadata -> steal role credentials

Category: infra-ssrf · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/09/metadata-ssrf` · **Defensive pattern:** `infra`

3.35.1 Challenge description

Review how the built-in challenge models ssrf to cloud metadata -> steal role credentials, focusing on its infra-ssrf trust boundary.

3.35.2 Educational focus

Workload identity and egress are authority boundaries; model-facing services should carry minimal ambient privilege.

Trust boundary: A model-facing workload crosses into cloud metadata, administrative APIs, registries, or network destinations.

3.35.3 Guided defensive review

1. Which destinations can the workload reach?
2. Which administrative or registry actions require authentication?

3.35.4 Defensive resolution

Default-deny egress, use workload identity, require admin authentication, isolate metadata, and keep secrets out of artifacts.

Validation: Review policy and use inert internal endpoints to confirm blocked destinations and authenticated administration.

Gotcha: Application input validation cannot compensate for broad network reach and over-privileged workload credentials.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.36 `09/open-inference-api` — Abuse an unauthenticated admin inference route

Category: infra-api · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/09/open-inference-api` · **Defensive pattern:** `infra`

3.36.1 Challenge description

Review how the built-in challenge models abuse an unauthenticated admin inference route, focusing on its infra-api trust boundary.

3.36.2 Educational focus

Workload identity and egress are authority boundaries; model-facing services should carry minimal ambient privilege.

Trust boundary: A model-facing workload crosses into cloud metadata, administrative APIs, registries, or network destinations.

3.36.3 Guided defensive review

1. Which destinations can the workload reach?
2. Which administrative or registry actions require authentication?

3.36.4 Defensive resolution

Default-deny egress, use workload identity, require admin authentication, isolate metadata, and keep secrets out of artifacts.

Validation: Review policy and use inert internal endpoints to confirm blocked destinations and authenticated administration.

Gotcha: Application input validation cannot compensate for broad network reach and over-privileged workload credentials.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.37 `09/registry-secrets` — Recover a secret from a model card

Category: infra-secrets · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/09/registry-secrets` · **Defensive pattern:** `secrets`

3.37.1 Challenge description

Review how the built-in challenge models recover a secret from a model card, focusing on its infra-secrets trust boundary.

3.37.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

3.37.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

3.37.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.38 `10/asset-inventory` — Inventory the high-value AI assets

Category: tm-assets · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/10/asset-inventory` · **Defensive pattern:** `threat`

3.38.1 Challenge description

Review how the built-in challenge models inventory the high-value ai assets, focusing on its tm-assets trust boundary.

3.38.2 Educational focus

A useful threat model explains relationships and complete paths rather than listing security keywords.

Trust boundary: Assets, principals, data flows, and effects cross one or more explicitly modelled trust boundaries.

3.38.3 Guided defensive review

1. Which asset and principal are involved?
2. What ordered path connects entry, privilege, action, and impact?

3.38.4 Defensive resolution

Record assets, trust boundaries, data flows, assumptions, ranked paths, owners, and controls tied to each boundary.

Validation: Review models for concrete actors and ordered relationships; reject disconnected keyword inventories.

Gotcha: A complete list of terms can still omit the relationship that makes a scenario possible.

Evidence status: Built-in artifact verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.39 `10/trust-boundaries` — Map trust boundaries & data flows

Category: tm-boundaries · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `builtin/10/trust-boundaries` · **Defensive pattern:** `threat`

3.39.1 Challenge description

Review how the built-in challenge models map trust boundaries & data flows, focusing on its tm-boundaries trust boundary.

3.39.2 Educational focus

A useful threat model explains relationships and complete paths rather than listing security keywords.

Trust boundary: Assets, principals, data flows, and effects cross one or more explicitly modelled trust boundaries.

3.39.3 Guided defensive review

1. Which asset and principal are involved?
2. What ordered path connects entry, privilege, action, and impact?

3.39.4 Defensive resolution

Record assets, trust boundaries, data flows, assumptions, ranked paths, owners, and controls tied to each boundary.

Validation: Review models for concrete actors and ordered relationships; reject disconnected keyword inventories.

Gotcha: A complete list of terms can still omit the relationship that makes a scenario possible.

Evidence status: Built-in artifact verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.40 `10/attack-paths` — Enumerate end-to-end attack paths

Category: tm-paths · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/10/attack-paths` · **Defensive pattern:** `threat`

3.40.1 Challenge description

Review how the built-in challenge models enumerate end-to-end attack paths, focusing on its tm-paths trust boundary.

3.40.2 Educational focus

A useful threat model explains relationships and complete paths rather than listing security keywords.

Trust boundary: Assets, principals, data flows, and effects cross one or more explicitly modelled trust boundaries.

3.40.3 Guided defensive review

1. Which asset and principal are involved?
2. What ordered path connects entry, privilege, action, and impact?

3.40.4 Defensive resolution

Record assets, trust boundaries, data flows, assumptions, ranked paths, owners, and controls tied to each boundary.

Validation: Review models for concrete actors and ordered relationships; reject disconnected keyword inventories.

Gotcha: A complete list of terms can still omit the relationship that makes a scenario possible.

Evidence status: Built-in artifact verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.41 `11/initial-access` — Initial access via indirect injection

Category: chain-access · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `builtin/11/initial-access` · **Defensive pattern:** `chain`

3.41.1 Challenge description

Review how the built-in challenge models initial access via indirect injection, focusing on its chain-access trust boundary.

3.41.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

3.41.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

3.41.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.42 `11/pivot-vault` — Pivot through ambient authority to the vault

Category: chain-pivot · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/11/pivot-vault` · **Defensive pattern:** `chain`

3.42.1 Challenge description

Review how the built-in challenge models pivot through ambient authority to the vault, focusing on its chain-pivot trust boundary.

3.42.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

3.42.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

3.42.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

3.43 `11/exfil-crown-jewel` — Exfiltrate the crown-jewel secret

Category: chain-exfil · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `builtin/11/exfil-crown-jewel` · **Defensive pattern:** `chain`

3.43.1 Challenge description

Review how the built-in challenge models exfiltrate the crown-jewel secret, focusing on its chain-exfil trust boundary.

3.43.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

3.43.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

3.43.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4 `dvaa` — Damn Vulnerable AI Agent

4.1 `L1-01` — Extract the System Prompt

Category: data-exfiltration · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `dvaa/L1-01` · **Defensive pattern:** `secrets`

4.1.1 Challenge description

Review the Extract the System Prompt scenario as implemented by OpenA2A, focusing on its data-exfiltration boundary.

4.1.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

4.1.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

4.1.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.2 `L1-02` — API Key Leak

Category: data-exfiltration · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `dvaa/L1-02` · **Defensive pattern:** `secrets`

4.2.1 Challenge description

Review the API Key Leak scenario as implemented by OpenA2A, focusing on its data-exfiltration boundary.

4.2.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

4.2.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

4.2.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.3 `L1-03` — Basic Prompt Injection

Category: prompt-injection · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `dvaa/L1-03` · **Defensive pattern:** `prompt`

4.3.1 Challenge description

Review the Basic Prompt Injection scenario as implemented by OpenA2A, focusing on its prompt-injection boundary.

4.3.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.3.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.3.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.4 `L2-01` — Jailbreak via Roleplay

Category: jailbreak · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-01` · **Defensive pattern:** `prompt`

4.4.1 Challenge description

Review the Jailbreak via Roleplay scenario as implemented by OpenA2A, focusing on its jailbreak boundary.

4.4.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.4.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.4.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.5 `L2-02` — Context Window Manipulation

Category: context-manipulation · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-02` · **Defensive pattern:** `prompt`

4.5.1 Challenge description

Review the Context Window Manipulation scenario as implemented by OpenA2A, focusing on its context-manipulation boundary.

4.5.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.5.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.5.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.6 L2-03 — MCP Path Traversal

Category: mcp-exploitation · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-03` · **Defensive pattern:** `sink`

4.6.1 Challenge description

Review the MCP Path Traversal scenario as implemented by OpenA2A, focusing on its mcp-exploitation boundary.

4.6.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

4.6.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

4.6.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.7 `L2-04` — Persistent Memory Injection

Category: memory-injection · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-04` · **Defensive pattern:** `prompt`

4.7.1 Challenge description

Review the Persistent Memory Injection scenario as implemented by OpenA2A, focusing on its memory-injection boundary.

4.7.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.7.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.7.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.8 L2-05 — Memory Credential Extraction

Category: memory-injection · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-05` · **Defensive pattern:** `secrets`

4.8.1 Challenge description

Review the Memory Credential Extraction scenario as implemented by OpenA2A, focusing on its memory-injection boundary.

4.8.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

4.8.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

4.8.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.9 `L2-06` — Context Padding Attack

Category: context-overflow · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-06` · **Defensive pattern:** `prompt`

4.9.1 Challenge description

Review the Context Padding Attack scenario as implemented by OpenA2A, focusing on its context-overflow boundary.

4.9.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.9.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.9.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.10 L2-07 — Safety Instruction Displacement

Category: context-overflow · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-07` · **Defensive pattern:** `prompt`

4.10.1 Challenge description

Review the Safety Instruction Displacement scenario as implemented by OpenA2A, focusing on its context-overflow boundary.

4.10.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.10.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.10.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.11 `L2-08` — Malicious Tool Registration

Category: tool-registry-poisoning · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-08` · **Defensive pattern:** `mcp`

4.11.1 Challenge description

Review the Malicious Tool Registration scenario as implemented by OpenA2A, focusing on its tool-registry-poisoning boundary.

4.11.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

4.11.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

4.11.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.12 `L2-09` — Tool Call MITM

Category: tool-mitm · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvaa/L2-09` · **Defensive pattern:** `transport`

4.12.1 Challenge description

Review the Tool Call MITM scenario as implemented by OpenA2A, focusing on its tool-mitm boundary.

4.12.2 Educational focus

Every layer must agree on one canonical request and bind credentials to the intended origin, audience, and session.

Trust boundary: Identity and request meaning cross proxies, origins, sessions, protocol versions, or downstream services.

4.12.3 Guided defensive review

1. Which layer decides request meaning?
2. Are origin, audience, version, and session identifiers bound exactly?

4.12.4 Defensive resolution

Canonicalise once, validate exact origins and audiences, use unpredictable sessions, and reject contradictory routing data.

Validation: Exercise mismatched headers and metadata with harmless requests and assert that every layer reaches the same deny decision.

Gotcha: A gateway and application can each appear correct while authorising different interpretations of one request.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.13 `L3-01` — Chained Prompt Injection

Category: prompt-injection · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvaa/L3-01` · **Defensive pattern:** `prompt`

4.13.1 Challenge description

Review the Chained Prompt Injection scenario as implemented by OpenA2A, focusing on its prompt-injection boundary.

4.13.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.13.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.13.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.14 L3-02 — SSRF via MCP

Category: mcp-exploitation · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvaa/L3-02` · **Defensive pattern:** `infra`

4.14.1 Challenge description

Review the SSRF via MCP scenario as implemented by OpenA2A, focusing on its mcp-exploitation boundary.

4.14.2 Educational focus

Workload identity and egress are authority boundaries; model-facing services should carry minimal ambient privilege.

Trust boundary: A model-facing workload crosses into cloud metadata, administrative APIs, registries, or network destinations.

4.14.3 Guided defensive review

1. Which destinations can the workload reach?
2. Which administrative or registry actions require authentication?

4.14.4 Defensive resolution

Default-deny egress, use workload identity, require admin authentication, isolate metadata, and keep secrets out of artifacts.

Validation: Review policy and use inert internal endpoints to confirm blocked destinations and authenticated administration.

Gotcha: Application input validation cannot compensate for broad network reach and over-privileged workload credentials.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.15 L3-03 — Self-Replicating Memory Entry

Category: memory-injection · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvaa/L3-03` · **Defensive pattern:** `prompt`

4.15.1 Challenge description

Review the Self-Replicating Memory Entry scenario as implemented by OpenA2A, focusing on its memory-injection boundary.

4.15.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.15.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.15.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.16 `L3-04` — System Prompt Extraction via Context Pressure

Category: context-overflow · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvaa/L3-04` · **Defensive pattern:** `prompt`

4.16.1 Challenge description

Review the System Prompt Extraction via Context Pressure scenario as implemented by OpenA2A, focusing on its context-overflow boundary.

4.16.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.16.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.16.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.17 `L3-05` — Tool Typosquatting

Category: tool-registry-poisoning · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvaa/L3-05` · **Defensive pattern:** `mcp`

4.17.1 Challenge description

Review the Tool Typosquatting scenario as implemented by OpenA2A, focusing on its tool-registry-poisoning boundary.

4.17.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

4.17.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

4.17.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.18 `L3-06` — Tool Chain Data Exfiltration

Category: tool-registry-poisoning · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvaa/L3-06` · **Defensive pattern:** `mcp`

4.18.1 Challenge description

Review the Tool Chain Data Exfiltration scenario as implemented by OpenA2A, focusing on its tool-registry-poisoning boundary.

4.18.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

4.18.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

4.18.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.19 L3-07 — Tool Shadowing

Category: tool-mitm · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvaa/L3-07` · **Defensive pattern:** `transport`

4.19.1 Challenge description

Review the Tool Shadowing scenario as implemented by OpenA2A, focusing on its tool-mitm boundary.

4.19.2 Educational focus

Every layer must agree on one canonical request and bind credentials to the intended origin, audience, and session.

Trust boundary: Identity and request meaning cross proxies, origins, sessions, protocol versions, or downstream services.

4.19.3 Guided defensive review

1. Which layer decides request meaning?
2. Are origin, audience, version, and session identifiers bound exactly?

4.19.4 Defensive resolution

Canonicalise once, validate exact origins and audiences, use unpredictable sessions, and reject contradictory routing data.

Validation: Exercise mismatched headers and metadata with harmless requests and assert that every layer reaches the same deny decision.

Gotcha: A gateway and application can each appear correct while authorising different interpretations of one request.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.20 L3-08 — Traffic Redirection Attack

Category: tool-mitm · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvaa/L3-08` · **Defensive pattern:** `transport`

4.20.1 Challenge description

Review the Traffic Redirection Attack scenario as implemented by OpenA2A, focusing on its tool-mitm boundary.

4.20.2 Educational focus

Every layer must agree on one canonical request and bind credentials to the intended origin, audience, and session.

Trust boundary: Identity and request meaning cross proxies, origins, sessions, protocol versions, or downstream services.

4.20.3 Guided defensive review

1. Which layer decides request meaning?
2. Are origin, audience, version, and session identifiers bound exactly?

4.20.4 Defensive resolution

Canonicalise once, validate exact origins and audiences, use unpredictable sessions, and reject contradictory routing data.

Validation: Exercise mismatched headers and metadata with harmless requests and assert that every layer reaches the same deny decision.

Gotcha: A gateway and application can each appear correct while authorising different interpretations of one request.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.21 `L4-01` — Compromise SecureBot

Category: mixed · **Difficulty:** expert · **Variants:** 1 **Catalog key:** `dvaa/L4-01` · **Defensive pattern:** `chain`

4.21.1 Challenge description

Review the Compromise SecureBot scenario as implemented by OpenA2A, focusing on its mixed boundary.

4.21.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

4.21.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

4.21.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

4.22 `L4-02` — Agent-to-Agent Attack Chain

Category: agent-to-agent · **Difficulty:** expert · **Variants:** 1 **Catalog key:** `dvaa/L4-02` · **Defensive pattern:** `identity`

4.22.1 Challenge description

Review the Agent-to-Agent Attack Chain scenario as implemented by OpenA2A, focusing on its agent-to-agent boundary.

4.22.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

4.22.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

4.22.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5 `mcpgoat` — MCPGoat

5.1 `A1` — Tool Poisoning

Category: MCP-specific · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/A1` · **Defensive pattern:** `mcp`

5.1.1 Challenge description

Review the Tool Poisoning scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

5.1.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

5.1.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

5.1.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.2 A2 — Tool Shadowing / Trusted-Tool Override

Category: MCP-specific · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/A2` · **Defensive pattern:** `mcp`

5.2.1 Challenge description

Review the Tool Shadowing / Trusted-Tool Override scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

5.2.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

5.2.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

5.2.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.3 A3 — Rug Pull / Tool Mutation (TOCTOU)

Category: MCP-specific · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/A3` · **Defensive pattern:** `mcp`

5.3.1 Challenge description

Review the Rug Pull / Tool Mutation (TOCTOU) scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

5.3.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

5.3.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

5.3.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.4 B1 — Indirect Prompt Injection via Tool Output

Category: Prompt/Context · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/B1` · **Defensive pattern:** `prompt`

5.4.1 Challenge description

Review the Indirect Prompt Injection via Tool Output scenario as implemented by Sabyasachi Dhal, focusing on its Prompt/Context boundary.

5.4.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.4.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.4.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.5 D1 — Command Injection (RCE)

Category: Injection sink · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/D1` · **Defensive pattern:** `sink`

5.5.1 Challenge description

Review the Command Injection (RCE) scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

5.5.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

5.5.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

5.5.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.6 D2 — Path Traversal / Arbitrary File Read

Category: Injection sink · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/D2` · **Defensive pattern:** `sink`

5.6.1 Challenge description

Review the Path Traversal / Arbitrary File Read scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

5.6.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

5.6.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

5.6.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.7 D3 — Server-Side Request Forgery

Category: Injection sink · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/D3` · **Defensive pattern:** `infra`

5.7.1 Challenge description

Review the Server-Side Request Forgery scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

5.7.2 Educational focus

Workload identity and egress are authority boundaries; model-facing services should carry minimal ambient privilege.

Trust boundary: A model-facing workload crosses into cloud metadata, administrative APIs, registries, or network destinations.

5.7.3 Guided defensive review

1. Which destinations can the workload reach?
2. Which administrative or registry actions require authentication?

5.7.4 Defensive resolution

Default-deny egress, use workload identity, require admin authentication, isolate metadata, and keep secrets out of artifacts.

Validation: Review policy and use inert internal endpoints to confirm blocked destinations and authenticated administration.

Gotcha: Application input validation cannot compensate for broad network reach and over-privileged workload credentials.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.8 D4 — SQL Injection

Category: Injection sink · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/D4` · **Defensive pattern:** `sink`

5.8.1 Challenge description

Review the SQL Injection scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

5.8.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

5.8.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

5.8.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.9 C2 — Broken Authorization / Confused Deputy

Category: AuthN/AuthZ · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/C2` · **Defensive pattern:** `identity`

5.9.1 Challenge description

Review the Broken Authorization / Confused Deputy scenario as implemented by Sabyasachi Dhal, focusing on its AuthN/AuthZ boundary.

5.9.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

5.9.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

5.9.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.10 C3 — Insecure Direct Object Reference

Category: AuthN/AuthZ · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/C3` · **Defensive pattern:** `identity`

5.10.1 Challenge description

Review the Insecure Direct Object Reference scenario as implemented by Sabyasachi Dhal, focusing on its AuthN/AuthZ boundary.

5.10.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

5.10.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

5.10.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.11 E1 — Sensitive Data Exposure

Category: Secrets/Exposure · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/E1` · **Defensive pattern:** `secrets`

5.11.1 Challenge description

Review the Sensitive Data Exposure scenario as implemented by Sabyasachi Dhal, focusing on its Secrets/Exposure boundary.

5.11.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

5.11.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

5.11.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.12 A9 — Invisible-Text Tool Poisoning

Category: MCP-specific · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/A9` · **Defensive pattern:** `mcp`

5.12.1 Challenge description

Review the Invisible-Text Tool Poisoning scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

5.12.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

5.12.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

5.12.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.13 B2 — Indirect Injection via Resource Content

Category: Prompt/Context · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/B2` · **Defensive pattern:** `prompt`

5.13.1 Challenge description

Review the Indirect Injection via Resource Content scenario as implemented by Sabyasachi Dhal, focusing on its Prompt/Context boundary.

5.13.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.13.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.13.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.14 B3 — Prompt-Template Injection

Category: Prompt/Context · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/B3` · **Defensive pattern:** `prompt`

5.14.1 Challenge description

Review the Prompt-Template Injection scenario as implemented by Sabyasachi Dhal, focusing on its Prompt/Context boundary.

5.14.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.14.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.14.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.15 B5 — Sampling Abuse (server-driven LLM calls)

Category: MCP-specific · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/B5` · **Defensive pattern:** `mcp`

5.15.1 Challenge description

Review the Sampling Abuse (server-driven LLM calls) scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

5.15.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

5.15.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

5.15.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.16 C4 — OAuth Token-Audience Confusion

Category: AuthN/AuthZ · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/C4` · **Defensive pattern:** `identity`

5.16.1 Challenge description

Review the OAuth Token-Audience Confusion scenario as implemented by Sabyasachi Dhal, focusing on its AuthN/AuthZ boundary.

5.16.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

5.16.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

5.16.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.17 D6 — Server-Side Template Injection

Category: Injection sink · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/D6` · **Defensive pattern:** `sink`

5.17.1 Challenge description

Review the Server-Side Template Injection scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

5.17.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

5.17.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

5.17.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.18 F1 — DNS Rebinding / Missing Origin Validation

Category: Transport · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/F1` · **Defensive pattern:** `transport`

5.18.1 Challenge description

Review the DNS Rebinding / Missing Origin Validation scenario as implemented by Sabyasachi Dhal, focusing on its Transport boundary.

5.18.2 Educational focus

Every layer must agree on one canonical request and bind credentials to the intended origin, audience, and session.

Trust boundary: Identity and request meaning cross proxies, origins, sessions, protocol versions, or downstream services.

5.18.3 Guided defensive review

1. Which layer decides request meaning?
2. Are origin, audience, version, and session identifiers bound exactly?

5.18.4 Defensive resolution

Canonicalise once, validate exact origins and audiences, use unpredictable sessions, and reject contradictory routing data.

Validation: Exercise mismatched headers and metadata with harmless requests and assert that every layer reaches the same deny decision.

Gotcha: A gateway and application can each appear correct while authorising different interpretations of one request.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.19 F2 — CORS Misconfiguration

Category: Transport · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/F2` · **Defensive pattern:** `transport`

5.19.1 Challenge description

Review the CORS Misconfiguration scenario as implemented by Sabyasachi Dhal, focusing on its Transport boundary.

5.19.2 Educational focus

Every layer must agree on one canonical request and bind credentials to the intended origin, audience, and session.

Trust boundary: Identity and request meaning cross proxies, origins, sessions, protocol versions, or downstream services.

5.19.3 Guided defensive review

1. Which layer decides request meaning?
2. Are origin, audience, version, and session identifiers bound exactly?

5.19.4 Defensive resolution

Canonicalise once, validate exact origins and audiences, use unpredictable sessions, and reject contradictory routing data.

Validation: Exercise mismatched headers and metadata with harmless requests and assert that every layer reaches the same deny decision.

Gotcha: A gateway and application can each appear correct while authorising different interpretations of one request.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.20 C6 — Predictable Session IDs (hijack)

Category: Transport · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/C6` · **Defensive pattern:** `transport`

5.20.1 Challenge description

Review the Predictable Session IDs (hijack) scenario as implemented by Sabyasachi Dhal, focusing on its Transport boundary.

5.20.2 Educational focus

Every layer must agree on one canonical request and bind credentials to the intended origin, audience, and session.

Trust boundary: Identity and request meaning cross proxies, origins, sessions, protocol versions, or downstream services.

5.20.3 Guided defensive review

1. Which layer decides request meaning?
2. Are origin, audience, version, and session identifiers bound exactly?

5.20.4 Defensive resolution

Canonicalise once, validate exact origins and audiences, use unpredictable sessions, and reject contradictory routing data.

Validation: Exercise mismatched headers and metadata with harmless requests and assert that every layer reaches the same deny decision.

Gotcha: A gateway and application can each appear correct while authorising different interpretations of one request.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.21 `G1` — Unbounded Consumption (cost/DoS)

Category: DoS/Cost · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/G1` · **Defensive pattern:** `dos`

5.21.1 Challenge description

Review the Unbounded Consumption (cost/DoS) scenario as implemented by Sabyasachi Dhal, focusing on its DoS/Cost boundary.

5.21.2 Educational focus

Availability and spend are security properties that require explicit per-request and aggregate budgets.

Trust boundary: Attacker-controlled work crosses compute, memory, token, concurrency, or financial budget boundaries.

5.21.3 Guided defensive review

1. Which resource grows with input?
2. Where are tenant, tool, and global budgets enforced?

5.21.4 Defensive resolution

Set timeouts, size and token limits, concurrency quotas, rate limits, cancellation, and cost telemetry.

Validation: Use bounded synthetic workloads to assert predictable rejection before resource use approaches operational limits.

Gotcha: Per-request limits alone miss distributed or repeated consumption across sessions and identities.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.22 G4 — Regular-Expression DoS (ReDoS)

Category: DoS/Cost · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/G4` · **Defensive pattern:** `dos`

5.22.1 Challenge description

Review the Regular-Expression DoS (ReDoS) scenario as implemented by Sabyasachi Dhal, focusing on its DoS/Cost boundary.

5.22.2 Educational focus

Availability and spend are security properties that require explicit per-request and aggregate budgets.

Trust boundary: Attacker-controlled work crosses compute, memory, token, concurrency, or financial budget boundaries.

5.22.3 Guided defensive review

1. Which resource grows with input?
2. Where are tenant, tool, and global budgets enforced?

5.22.4 Defensive resolution

Set timeouts, size and token limits, concurrency quotas, rate limits, cancellation, and cost telemetry.

Validation: Use bounded synthetic workloads to assert predictable rejection before resource use approaches operational limits.

Gotcha: Per-request limits alone miss distributed or repeated consumption across sessions and identities.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.23 D5 — NoSQL Injection

Category: Injection sink · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/D5` · **Defensive pattern:** `sink`

5.23.1 Challenge description

Review the NoSQL Injection scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

5.23.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

5.23.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

5.23.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.24 D7 — XML External Entity (XXE)

Category: Injection sink · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/D7` · **Defensive pattern:** `sink`

5.24.1 Challenge description

Review the XML External Entity (XXE) scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

5.24.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

5.24.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

5.24.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.25 D8 — Insecure Deserialization (prototype pollution)

Category: Injection sink · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/D8` · **Defensive pattern:** `sink`

5.25.1 Challenge description

Review the Insecure Deserialization (prototype pollution) scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

5.25.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

5.25.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

5.25.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

5.26 `H1` — Supply Chain (typosquat / unsigned package)

Category: Supply chain · **Difficulty:** tiered · **Variants:** 3 **Catalog key:** `mcpgoat/H1` · **Defensive pattern:** `supply`

5.26.1 Challenge description

Review the Supply Chain (typosquat / unsigned package) scenario as implemented by Sabyasachi Dhal, focusing on its Supply chain boundary.

5.26.2 Educational focus

AI artifacts can be executable or behavior-changing inputs and need the same provenance controls as software.

Trust boundary: A model artifact, dataset, adapter, dependency, or registry record crosses the software supply-chain boundary.

5.26.3 Guided defensive review

1. What identity and digest bind the artifact?
2. Where is it loaded, merged, trained, or resolved?

5.26.4 Defensive resolution

Pin immutable digests, verify signatures and provenance, use safe formats, scan dependencies, and load in isolation.

Validation: Confirm allow-listed sources and digests with benign fixtures, then fail closed on unknown or mutable artifacts.

Gotcha: A familiar filename, package name, or model label is not proof of origin or safe behavior.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6 `dvmcp` — Damn Vulnerable MCP Server

6.1 `01` — Basic Prompt Injection

Category: Prompt/Context · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `dvmcp/01` · **Defensive pattern:** `prompt`

6.1.1 Challenge description

Review the Basic Prompt Injection scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Prompt/Context boundary.

6.1.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

6.1.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

6.1.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.2 02 — Tool Poisoning

Category: MCP-specific · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `dvmcp/02` · **Defensive pattern:** `mcp`

6.2.1 Challenge description

Review the Tool Poisoning scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its MCP-specific boundary.

6.2.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

6.2.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

6.2.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.3 03 — Excessive Permission Scope

Category: AuthN/AuthZ · **Difficulty:** easy · **Variants:** 1 **Catalog key:** `dvmcp/03` · **Defensive pattern:** `identity`

6.3.1 Challenge description

Review the Excessive Permission Scope scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its AuthN/AuthZ boundary.

6.3.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

6.3.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

6.3.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.4 04 — Rug Pull Attack

Category: MCP-specific · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvmcp/04` · **Defensive pattern:** `mcp`

6.4.1 Challenge description

Review the Rug Pull Attack scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its MCP-specific boundary.

6.4.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

6.4.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

6.4.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.5 05 — Tool Shadowing

Category: MCP-specific · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvmcp/05` · **Defensive pattern:** `mcp`

6.5.1 Challenge description

Review the Tool Shadowing scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its MCP-specific boundary.

6.5.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

6.5.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

6.5.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.6 06 — Indirect Prompt Injection

Category: Prompt/Context · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvmcp/06` · **Defensive pattern:** `prompt`

6.6.1 Challenge description

Review the Indirect Prompt Injection scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Prompt/Context boundary.

6.6.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

6.6.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

6.6.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.7 07 — Token Theft

Category: Secrets/Exposure · **Difficulty:** medium · **Variants:** 1 **Catalog key:** `dvmcp/07` · **Defensive pattern:** `secrets`

6.7.1 Challenge description

Review the Token Theft scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Secrets/Exposure boundary.

6.7.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

6.7.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

6.7.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.8 08 — Malicious Code Execution

Category: Injection sink · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvmcp/08` · **Defensive pattern:** `sink`

6.8.1 Challenge description

Review the Malicious Code Execution scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Injection sink boundary.

6.8.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

6.8.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

6.8.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.9 09 — Remote Access Control

Category: Injection sink · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvmcp/09` · **Defensive pattern:** `sink`

6.9.1 Challenge description

Review the Remote Access Control scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Injection sink boundary.

6.9.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

6.9.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

6.9.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

6.10 10 — Multi-Vector Attack

Category: Attack chain · **Difficulty:** hard · **Variants:** 1 **Catalog key:** `dvmcp/10` · **Defensive pattern:** `chain`

6.10.1 Challenge description

Review the Multi-Vector Attack scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Attack chain boundary.

6.10.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

6.10.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

6.10.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7 `appsecco-mcp` — Vulnerable MCP Servers Lab

7.1 `filesystem-workspace-actions` — Filesystem Workspace Actions

Category: Path traversal / code execution · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/filesystem-workspace-actions` · **Defensive pattern:** `sink`

7.1.1 Challenge description

Review the Filesystem Workspace Actions scenario as implemented by Appsecco, focusing on its Path traversal / code execution boundary.

7.1.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

7.1.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

7.1.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

7.2 `indirect-prompt-injection` — Indirect Prompt Injection (stdio)

Category: Prompt/Context · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/indirect-prompt-injection` · **Defensive pattern:** `prompt`

7.2.1 Challenge description

Review the Indirect Prompt Injection (stdio) scenario as implemented by Appsecco, focusing on its Prompt/Context boundary.

7.2.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

7.2.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

7.2.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

7.3 `indirect-prompt-injection-remote` — Indirect Prompt Injection (HTTP+SSE)

Category: Prompt/Context · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/indirect-prompt-injection-remote` · **Defensive pattern:** `prompt`

7.3.1 Challenge description

Review the Indirect Prompt Injection (HTTP+SSE) scenario as implemented by Appsecco, focusing on its Prompt/Context boundary.

7.3.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

7.3.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

7.3.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

7.4 `malicious-code-exec` — Malicious Code Execution

Category: Injection sink · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/malicious-code-exec` · **Defensive pattern:** `sink`

7.4.1 Challenge description

Review the Malicious Code Execution scenario as implemented by Appsecco, focusing on its Injection sink boundary.

7.4.2 Educational focus

The safe boundary is the sink: preserve structure and avoid turning data into executable syntax.

Trust boundary: Untrusted structured input reaches a filesystem, interpreter, query engine, parser, or process boundary.

7.4.3 Guided defensive review

1. Which sink interprets the value?
2. Is validation applied after decoding and canonicalisation?

7.4.4 Defensive resolution

Use parameterised APIs, safe parsers, canonical path containment, allow-lists, and a restricted runtime.

Validation: Add non-destructive negative tests for malformed, encoded, absolute, and boundary-crossing values.

Gotcha: A blacklist applied before decoding or normalisation can approve a value that becomes dangerous later.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

7.5 `malicious-tools` — Malicious Tools

Category: Tool output poisoning · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/malicious-tools` · **Defensive pattern:** `mcp`

7.5.1 Challenge description

Review the Malicious Tools scenario as implemented by Appsecco, focusing on its Tool output poisoning boundary.

7.5.2 Educational focus

Treat server metadata and results as untrusted input, and keep protocol capability separate from permission.

Trust boundary: MCP metadata, content, or a remote capability crosses from a server into a host with model or system authority.

7.5.3 Guided defensive review

1. What does the host trust before execution?
2. Can definitions, results, or capabilities change after approval?

7.5.4 Defensive resolution

Pin and diff definitions, canonicalise metadata, authorise each call, sandbox execution, and require explicit consent.

Validation: Validate canonical definitions and repeated calls with benign fixtures; assert that changed or excessive capability is rejected.

Gotcha: One-time approval does not cover later metadata mutation, hidden text, or a broader runtime effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

7.6 `namespace-typosquatting` — Namespace Typosquatting

Category: Supply chain · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/namespace-typosquatting` · **Defensive pattern:** `supply`

7.6.1 Challenge description

Review the Namespace Typosquatting scenario as implemented by Appsecco, focusing on its Supply chain boundary.

7.6.2 Educational focus

AI artifacts can be executable or behavior-changing inputs and need the same provenance controls as software.

Trust boundary: A model artifact, dataset, adapter, dependency, or registry record crosses the software supply-chain boundary.

7.6.3 Guided defensive review

1. What identity and digest bind the artifact?
2. Where is it loaded, merged, trained, or resolved?

7.6.4 Defensive resolution

Pin immutable digests, verify signatures and provenance, use safe formats, scan dependencies, and load in isolation.

Validation: Confirm allow-listed sources and digests with benign fixtures, then fail closed on unknown or mutable artifacts.

Gotcha: A familiar filename, package name, or model label is not proof of origin or safe behavior.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

7.7 `outdated-packages` — Outdated Packages

Category: Supply chain · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/outdated-packages` · **Defensive pattern:** `supply`

7.7.1 Challenge description

Review the Outdated Packages scenario as implemented by Appsecco, focusing on its Supply chain boundary.

7.7.2 Educational focus

AI artifacts can be executable or behavior-changing inputs and need the same provenance controls as software.

Trust boundary: A model artifact, dataset, adapter, dependency, or registry record crosses the software supply-chain boundary.

7.7.3 Guided defensive review

1. What identity and digest bind the artifact?
2. Where is it loaded, merged, trained, or resolved?

7.7.4 Defensive resolution

Pin immutable digests, verify signatures and provenance, use safe formats, scan dependencies, and load in isolation.

Validation: Confirm allow-listed sources and digests with benign fixtures, then fail closed on unknown or mutable artifacts.

Gotcha: A familiar filename, package name, or model label is not proof of origin or safe behavior.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

7.8 `secrets-pii` — Secrets and PII Exposure

Category: Secrets/Exposure · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/secrets-pii` · **Defensive pattern:** `secrets`

7.8.1 Challenge description

Review the Secrets and PII Exposure scenario as implemented by Appsecco, focusing on its Secrets/Exposure boundary.

7.8.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

7.8.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

7.8.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

7.9 `wikipedia-streamable` — Wikipedia (Streamable HTTP)

Category: Remote content injection · **Difficulty:** ungraded · **Variants:** 1 **Catalog key:** `appsecco-mcp/wikipedia-streamable` · **Defensive pattern:** `prompt`

7.9.1 Challenge description

Review the Wikipedia (Streamable HTTP) scenario as implemented by Appsecco, focusing on its Remote content injection boundary.

7.9.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

7.9.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

7.9.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e